REVISION CHEAT SHEET:
DATABASE • DBMS • RDBMS
MICROSOFT SQL SERVER

Interview-ready notes • examples • SQL • diagrams

Database

A database is an organized collection of related data that is stored electronically so that it can be easily accessed, managed, updated, and maintained.

In simple terms, a database is a place where an application stores the information it needs to keep for future use.

For example, an application may need to store information such as:

EXAMPLE / FLOW

Customers
Employees
Products
Orders
Payments

Instead of keeping all this information randomly, a database stores it in an organized and structured way.

Why a Database Is Used

Applications usually need to store data permanently.

For example, if a customer creates an account, the application should still remember that customer even after the application is closed or the server is restarted.

A database provides this permanent storage.

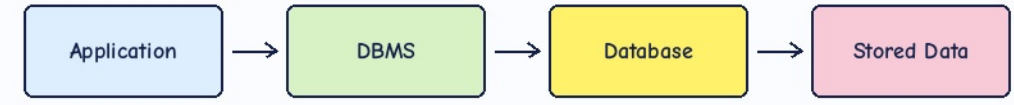
EXAMPLE / FLOW

Application
v
Database
v
Stored Data

The database allows the application to later retrieve, update, or delete that stored information.

DBMS - Database Management System

DATABASE + DBMS — COMPLETE PICTURE



Database = organized stored data

A DBMS stands for Database Management System.

A DBMS is software that is used to create, store, manage, retrieve, update, and control data inside databases.

In simple terms:

A DBMS is the software that helps users and applications work with a database.

For example:

EXAMPLE / FLOW

Application
v
DBMS
v
Database

The application normally does not manage the database files directly. It communicates with the DBMS, and the DBMS manages the actual data.

Examples of DBMS software include:

EXAMPLE / FLOW

Microsoft SQL Server
MySQL
PostgreSQL
Oracle Database
SQLite

#

Why Do We Need a DBMS?

Before database management systems were commonly used, applications often stored data directly in files.

For example:

EXAMPLE / FLOW

customers.txt
orders.txt

EXAMPLE / FLOW (cont.)

products.txt

This can work for very small applications, but it becomes difficult when the amount of data increases.

Some common problems with simple file-based storage are:

- Duplicate data
- Difficult searching
- Difficult updates
- Poor security
- Data inconsistency
- Multiple users modifying the same data
- Difficult backup and recovery
- Difficult relationship management

A DBMS is used to solve many of these problems.

Instead of every application directly managing files:

EXAMPLE / FLOW

Application
v
Files

we use:

EXAMPLE / FLOW

Application
v
DBMS
v
Database

The DBMS becomes responsible for managing the data properly.

What Does a DBMS Do?

A DBMS performs many important operations.

#

Creating Databases

A DBMS allows us to create databases.

For example, in SQL Server:

```sql
CREATE DATABASE ShoppingDb;
```

Here, SQL Server is the DBMS.

ShoppingDb is the database.

#

Storing Data

The DBMS stores data inside the database.

For example, customer information such as:

EXAMPLE / FLOW

CustomerId
Name
Email
Phone

can be stored and managed by the DBMS.

#

Retrieving Data

A DBMS allows users and applications to retrieve stored information.

Example:

```sql
SELECT *
FROM Customers;
```

The DBMS processes the query and returns the requested data.

#

Updating Data

Existing information can be changed.

Example:

```sql
UPDATE Customers
SET Name = 'John Smith'
WHERE CustomerId = 1;
```

The DBMS finds the correct record and updates it.

#

Deleting Data

A DBMS can also remove data.

Example:

```sql
DELETE FROM Customers
WHERE CustomerId = 1;
```

#

Managing Security

A DBMS helps control who can access the data.

For example:

```
Admin
-> Can read and modify data

Employee
-> Can read some data

Guest
-> Very limited access
```

This helps protect sensitive information.

#

Maintaining Data Consistency

A DBMS helps prevent incorrect or inconsistent data.

For example, an application may require:

```
CustomerId must be unique
Email cannot be empty
Price cannot be negative
```

The DBMS can enforce such rules.

#

Managing Multiple Users

In real applications, many users may access the same database at the same time.

For example:

EXAMPLE / FLOW

User 1
User 2
User 3
User 4
v
DBMS
v
Database

The DBMS helps manage these simultaneous operations safely.

#

Backup and Recovery

A DBMS can provide backup and recovery features.

If something goes wrong, such as:

EXAMPLE / FLOW

Hardware failure
Application failure
Accidental data loss

the database may be restored using backups and recovery mechanisms.

Simple Example

Suppose we have an online shopping application.

The application needs to store:

EXAMPLE / FLOW

Customers
Products
Orders
Payments

The structure may look like:

EXAMPLE / FLOW

Shopping Application
v
Microsoft SQL Server
v
ShoppingDb

Here:

EXAMPLE / FLOW

Microsoft SQL Server = DBMS

ShoppingDb = Database

SQL Server manages the database and provides the application with access to its data.

DBMS vs Database

This difference is very important.

A database is the actual organized collection of data.

A DBMS is the software used to manage that database.

For example:

EXAMPLE / FLOW

SQL Server
v
ShoppingDb

Here:

EXAMPLE / FLOW

SQL Server = DBMS

ShoppingDb = Database

A simple way to remember this is:

EXAMPLE / FLOW

DBMS -> Software
Database -> Stored Data

Advantages of a DBMS

A DBMS provides many benefits.

#

Organized Data Management

It provides a structured way to store and manage data.

#

Reduces Duplicate Data

A properly designed database can reduce unnecessary duplication.

#

Better Data Consistency

Rules can be used to keep data correct and consistent.

#

Security

Access to data can be controlled.

#

Easy Data Retrieval

Users can search and retrieve required information efficiently.

#

Multiple User Support

Many users can work with the database at the same time.

#

Backup and Recovery

Data can be backed up and recovered when necessary.

#

Better Data Control

The DBMS centrally manages how applications interact with the stored data.

Disadvantages of a DBMS

A DBMS also has some limitations.

#

Requires Knowledge

Developers and administrators need to understand database concepts.

#

Can Be Complex

Large database systems can require proper configuration and management.

#

Resource Usage

A DBMS may require:

EXAMPLE / FLOW

Memory
CPU
Storage
Server resources

#

Maintenance

Production databases may require:

EXAMPLE / FLOW

Backups
Monitoring
Security
Updates
Performance tuning

#

Cost

Some commercial database systems may require licensing or infrastructure costs.

Types of DBMS

DBMS systems can be classified based on how they organize data.

#

Hierarchical DBMS

In a hierarchical DBMS, data is organized like a tree.

Example:

EXAMPLE / FLOW

Company
v
Department
v
Employee

Each child usually has one parent.

This model was more common in older database systems.

#

Network DBMS

A network DBMS allows records to have multiple relationships.

For example:

EXAMPLE / FLOW

Student
\
Course
/
Teacher

This model provides more flexible relationships than the hierarchical model.

#

Relational DBMS

A relational DBMS stores data mainly in tables containing rows and columns.

Example:

EXAMPLE / FLOW

Customers

CustomerId	Name
1	John
2	Priya

And:

EXAMPLE / FLOW

Orders

OrderId	CustomerId
101	1
102	2

The tables can be related through common values such as CustomerId.

Examples of relational DBMS software include:

EXAMPLE / FLOW

Microsoft SQL Server
PostgreSQL
MySQL
Oracle Database

SQL Server belongs to this category.

We will study RDBMS separately, so you do not need to go deeply into it here.

#

Object-Oriented DBMS

An object-oriented DBMS stores data using object-oriented concepts.

The stored data can represent objects containing both:

EXAMPLE / FLOW

Data
+
Object-related structure

These systems are less common than relational databases for general business applications.

When Is a DBMS Used?

A DBMS is useful when an application needs to store and manage significant amounts of persistent data.

Examples include systems for:

EXAMPLE / FLOW

Banking
Shopping
Healthcare
Education
Payroll
Inventory
Customer management
Order processing

A DBMS is especially useful when applications need:

- Persistent data
- Data security
- Multiple-user access
- Data consistency
- Searching
- Updating
- Backup and recovery

When Might a DBMS Not Be Needed?

A full DBMS may not be necessary for a very small program that only works with temporary data.

For example:

EXAMPLE / FLOW

```
int firstNumber = 10;
int secondNumber = 20;
```

EXAMPLE / FLOW (cont.)

```
int total = firstNumber + secondNumber;
```

If total is needed only while the program is running, a database and DBMS may not be necessary.

Important Points to Remember

A DBMS is software used to manage databases.

Its main responsibilities include:

EXAMPLE / FLOW

Create
Store
Read
Update
Delete
Secure
Manage
Backup
Recover

The most important distinction is:

EXAMPLE / FLOW

DBMS = Software

Database = Organized collection of data

For your SQL Server learning:

EXAMPLE / FLOW

Microsoft SQL Server = DBMS

EmployeeDb
ShoppingDb
FundraisingDb
= Databases

#

Interview Definition

A DBMS, or Database Management System, is software used to create, store, retrieve, update, secure, and manage data inside databases while providing controlled access to users and applications.

Think of it this way:

Database = the data. DBMS = the software that manages that data.

A very simple analogy is:

EXAMPLE / FLOW

```
Database = Book
DBMS     = Librarian
```

The book contains the information. The librarian helps you find, add, organize, protect, and manage the books.

In SQL Server terms:

Microsoft SQL Server = DBMS

EXAMPLE / FLOW

```
ShoppingDb          = Database
EmployeeDb          = Database
BankDb              = Database
```

So if you install Microsoft SQL Server, you installed a DBMS.

Inside SQL Server, you can create databases:

SQL

```
CREATE DATABASE ShoppingDb;
```

SQL

```
CREATE DATABASE EmployeeDb;
```

Now you have:

EXAMPLE / FLOW

```
Microsoft SQL Server
v
DBMS
v
+---------------+
| ShoppingDb    |
| EmployeeDb    |
| BankDb        |
+---------------+
Databases
```

Another easy comparison:

EXAMPLE / FLOW

```
Database⊠DBMS
Stores the actual data⊠Manages the database
Contains application information⊠Software
Example: ShoppingDb⊠Example: Microsoft SQL Server
```

Contains customers, products, orders, etc.Lets us create, read, update, delete, secure, and manage that data

For example, imagine this customer data:

EXAMPLE / FLOW

CustomerId: 1
Name: John
Email: john@gmail.com

That information is stored in a:

Database

But when you run:

SQL

```
SELECT * FROM Customers;
```

the DBMS processes that command and gets the data from the database.

So:

EXAMPLE / FLOW

You
v
SQL Command
v
DBMS
v
Database
v
Data

The easiest sentence to remember is:

A database stores the data. A DBMS is the software used to manage the database.

For your learning:

SQL Server = DBMS

MyApplicationDb = Database

That is the main difference.

Microsoft SQL Server

Microsoft SQL Server is a database management system developed by Microsoft. More specifically, it is a Relational Database Management System (RDBMS) used to store, organize, retrieve, update, secure, and manage application data.

In simple words:

SQL Server is software from Microsoft that applications use to store and manage data in databases.

For example, suppose you install Microsoft SQL Server and create a database called:

EXAMPLE / FLOW

EmployeeDb

Then:

EXAMPLE / FLOW

Microsoft SQL Server -> Software that manages the data
EmployeeDb -> Database containing the data

You could visualize it simply as:

EXAMPLE / FLOW

```
Microsoft SQL Server
|
+-- EmployeeDb
+-- ShoppingDb
+-- FundraisingDb
```

SQL Server accepts commands written mainly in T-SQL (Transact-SQL) to work with data. Microsoft officially describes SQL Server as an RDBMS and states that applications communicate with it using T-SQL.

The current major release is SQL Server 2025 (17.x). Microsoft released SQL Server 2025 on November 18, 2025, and it is the current generation as of September 2026.

#

Why Do We Use SQL Server?

Imagine that you are developing an application that needs to save customers.

A customer may have:

EXAMPLE / FLOW

CustomerId
FirstName
LastName
Email
Phone

You need somewhere to permanently store that information.

SQL Server can store it inside a database.

For example:

```sql
CREATE DATABASE CustomerDb;
GO
```

Inside that database, you can create structures for storing customer information and later perform operations such as:

```sql
SELECT *
FROM Customers;
```

SQL Server processes the request, finds the information, and returns it to your application.

So the basic idea is:

EXAMPLE / FLOW

```
Application
v
SQL Server
v
Database
v
Stored Data
```

SQL Server is used because storing data is only one part of the problem. Real applications also need to search, update, protect, validate, recover, and process that data efficiently.

What Can SQL Server Do?

SQL Server provides much more than simply saving information.

It can help an application:

- Store data permanently
- Retrieve required data
- Insert new data
- Update existing data
- Delete data
- Search millions of records
- Protect sensitive information
- Control which users can access data
- Handle many users at the same time
- Maintain data consistency
- Perform transactions
- Create backups
- Recover data after failures
- Improve query performance
- Process relational data

- Work with JSON and XML
- Perform reporting and analytical workloads

The SQL Server Database Engine is Microsoft's core service responsible for storing, processing, and securing data.

Simple SQL Server Example

Suppose you want a database for employees.

SQL

Create the database:

```sql
CREATE DATABASE EmployeeDb;
GO
```

SQL

Use it:

```sql
USE EmployeeDb;
GO
```

Then your application can eventually store information such as:

EXAMPLE / FLOW

```
EmployeeId    FirstName    Department
-----------------------------------------------
1             John         IT
2             Priya        Finance
3             David        HR
```

SQL Server manages that information for you.

The important point here is not the table syntax yet. It is simply understanding that:

EXAMPLE / FLOW

```
SQL Server
v
EmployeeDb
v
Employee information
```

SQL Server and SQL Are Not the Same Thing

This is an important beginner distinction.

SQL is a language used to communicate with relational databases.

For example:

```sql
SELECT *
FROM Employees;
```

SQL Server is Microsoft's database-management software that understands SQL and Microsoft's extended SQL language called T-SQL.

So:

EXAMPLE / FLOW

SQL = Language

SQL Server = Database management software

Similar to:

EXAMPLE / FLOW

C

= Programming language

.NET = Platform that can execute and support C

applications

They are related, but they are not the same thing.

Why Is SQL Server Popular?

SQL Server is widely used for business and enterprise applications because it combines database storage with strong security, transaction support, performance features, administration tools, development tools, and integration with Microsoft's ecosystem.

It is especially common in organizations already using technologies such as:

EXAMPLE / FLOW

.NET
C

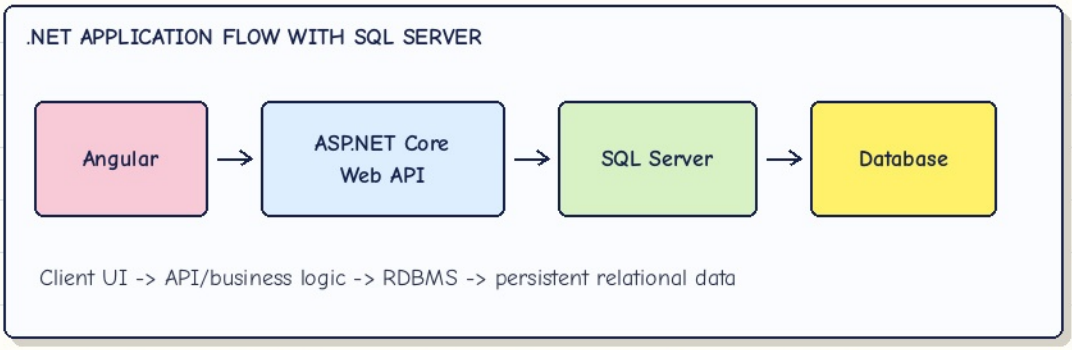
#
ASP.NET Core
Windows
Azure
Power BI
Microsoft Entra ID
Microsoft development tools

This does not mean SQL Server can only be used with Microsoft technologies.

For example, SQL Server can run on both Windows and Linux, as well as Linux containers and virtual machines.

Applications written in many different programming languages can connect to SQL Server.

Why SQL Server Is Very Good for .NET Applications



For your learning path, this is one of the biggest reasons SQL Server is useful.

You are learning:

EXAMPLE / FLOW

C

#
.NET
ASP.NET Core Web API
Angular
Azure

SQL Server works extremely well in this ecosystem.

For example:

EXAMPLE / FLOW

Angular
v
ASP.NET Core Web API
v
SQL Server

Your Web API could receive:

EXAMPLE / FLOW

GET /api/employees

and eventually retrieve information from SQL Server.

Microsoft provides mature SQL Server drivers and tooling for .NET, and technologies such as Entity Framework Core work naturally with SQL Server.

That makes SQL Server a very common choice for enterprise .NET applications.

Advantages of SQL Server

#

Strong Data Reliability

SQL Server is designed for applications where losing or corrupting business data would be serious.

Examples include:

EXAMPLE / FLOW

Banking information
Payments
Orders
Employees
Healthcare records
Insurance information
Business transactions

It provides mechanisms for maintaining consistency and recovering data when problems occur.

#

Strong Transaction Support

Many business operations contain multiple steps that must succeed together.

For example:

EXAMPLE / FLOW

Transfer $100

Account A -> Remove $100
Account B -> Add $100

You do not want only the first operation to succeed.

SQL Server provides transaction support for these kinds of operations.

This makes it particularly useful for financial and business applications.

#

Excellent .NET Integration

SQL Server has excellent integration with Microsoft's development ecosystem.

For a stack such as:

EXAMPLE / FLOW

ASP.NET Core
C

EXAMPLE / FLOW (cont.)

#
Entity Framework Core
SQL Server
Azure

the developer experience is mature and well supported.

This is one of the reasons many companies building .NET applications choose SQL Server.

#

Strong Security

SQL Server provides many security capabilities for controlling database access and protecting information.

You can control things such as:

EXAMPLE / FLOW

Who can connect?
Who can read data?
Who can insert data?
Who can modify data?
Who can delete data?
Who can administer the server?

It also includes enterprise security capabilities such as encryption and detailed permission management.

#

Good Performance

SQL Server is capable of handling large business workloads when the database and queries are designed correctly.

It contains sophisticated systems for:

EXAMPLE / FLOW

Query optimization
Indexing
Caching
Parallel processing
Query execution
Memory management

You do not need to understand those yet. We will learn them separately.

The important point is that SQL Server is designed for serious database workloads.

#

Excellent Management Tools

SQL Server has mature tools for developers and database administrators.

One of the most widely used is:

EXAMPLE / FLOW

SQL Server Management Studio
SSMS

It allows you to:

EXAMPLE / FLOW

Connect to SQL Server
Create databases
Write queries
View data
Manage security
Monitor databases
Backup databases
Troubleshoot problems

Microsoft lists SSMS as supported across SQL Server Enterprise, Standard, and Express editions.

#

Good Enterprise Support

Microsoft provides long-term servicing and support for SQL Server.

For example, SQL Server 2025 has mainstream support scheduled through January 7, 2031, with extended support through January 7, 2036.

This matters to large companies because business applications may remain in production for many years.

#

Works On-Premises and in the Cloud

SQL Server can run in different environments.

For example:

EXAMPLE / FLOW

Your own server
Windows server
Linux server
Virtual machine
Container
Azure Virtual Machine

That gives organizations flexibility about where they run their databases.

#

Supports More Than Basic Table Data

Modern SQL Server isn't limited to simple rows and columns.

SQL Server 2025 supports features involving:

EXAMPLE / FLOW

Relational data
JSON
XML
Full-text search
Graph data
Temporal data
Regular expressions
Vector/AI-related workloads

For example, SQL Server 2025 includes a native JSON data type and JSON indexing capabilities.

These features can become useful later in modern application and AI development.

#

Free Editions Are Available for Learning

You do not necessarily have to purchase SQL Server simply to learn it.

Microsoft provides development-oriented editions.

#

Developer Edition

Developer editions provide SQL Server capabilities for:

EXAMPLE / FLOW

Development
Testing
Learning

but are not licensed for production use.

SQL Server 2025 provides Enterprise Developer and Standard Developer editions.

For your learning, Developer Edition is an excellent choice.

#

Express Edition

SQL Server Express is free and intended for learning and smaller applications.

SQL Server 2025 Express allows a maximum relational database size of 50 GB, an increase from earlier releases.

Disadvantages of SQL Server

SQL Server is powerful, but it is not automatically the best database for every application.

#

Licensing Can Be Expensive

Developer and Express editions are free for their permitted purposes, but production editions such as Standard and Enterprise involve commercial licensing.

For large deployments, licensing cost can become an important consideration.

This is one area where open-source databases such as PostgreSQL and MySQL can have an advantage.

#

Can Require Significant Resources

A production SQL Server installation can require substantial:

EXAMPLE / FLOW

CPU
Memory
Disk
Server resources

depending on the application workload.

For a very small application, installing and operating a full SQL Server instance may be unnecessary.

#

Administration Can Become Complex

Enterprise SQL Server environments can require people who understand:

EXAMPLE / FLOW

Performance
Security
Backups
Recovery
High availability
Monitoring
Indexing
Configuration

A poorly managed SQL Server environment can still experience performance and reliability problems.

#

Microsoft Ecosystem Dependency Can Be a Consideration

SQL Server supports many environments and runs on both Windows and Linux, so it is no longer Windows-only.

However, organizations heavily using SQL Server commonly become connected to Microsoft's database tools, licensing, and ecosystem.

Some companies deliberately prefer open-source database technologies to reduce this dependency.

#

Not Always Necessary for Very Small Applications

Suppose you are building a tiny desktop program that stores only a few local settings.

Using a large SQL Server installation may be unnecessary.

Something lightweight such as SQLite could make more sense.

So:

EXAMPLE / FLOW

Small local application
v
SQL Server may be excessive

Whereas:

EXAMPLE / FLOW

Large business application
v
SQL Server may be an excellent choice

Is SQL Server the Best Database?

You should not memorize that SQL Server is always the best database.

There is no database that is best for every application.

A better statement is:

SQL Server is one of the strongest choices for enterprise relational applications, particularly when using .NET and Microsoft's ecosystem.

The correct database depends on:

EXAMPLE / FLOW

Application requirements
Budget
Existing technology stack
Team knowledge

EXAMPLE / FLOW (cont.)

Performance requirements
Data type
Infrastructure
Cloud provider
Licensing requirements
Scale

SQL Server Compared With Other Popular Databases

A simple high-level comparison is enough at this stage.

Database	Main Strength
SQL Server	Excellent enterprise database, especially strong with .NET and Microsoft technologies
PostgreSQL	Powerful open-source relational database with strong standards and extensibility
MySQL	Popular open-source database, commonly used for web applications
Oracle Database	Powerful enterprise database commonly used in very large organizations
SQLite	Very lightweight embedded database useful for local applications

This does not mean one is automatically better than another.

#

SQL Server vs PostgreSQL

Both are powerful relational database systems.

SQL Server is often attractive when your organization uses:

EXAMPLE / FLOW

.NET
Azure
Microsoft enterprise technologies
Microsoft administration tools

PostgreSQL is attractive when organizations prefer:

EXAMPLE / FLOW

Open-source software
No commercial database licensing
Strong extensibility
Cross-platform environments

For many applications, either one can be an excellent technical choice.

#

SQL Server vs MySQL

MySQL has historically been very popular for web applications and open-source stacks.

SQL Server generally provides a very integrated enterprise experience around:

EXAMPLE / FLOW

Microsoft technologies
.NET development
Enterprise administration
Business intelligence
Security
Microsoft tooling

MySQL may be attractive when an application needs a popular open-source relational database with relatively straightforward deployment.

#

SQL Server vs Oracle

Both are major enterprise database platforms.

Oracle Database is heavily used by many large enterprises and supports extremely demanding systems.

SQL Server can be particularly attractive to organizations using the Microsoft ecosystem because its development and administration tools integrate naturally with those technologies.

Both commercial enterprise deployments can involve significant licensing costs.

#

SQL Server vs SQLite

These databases serve very different purposes.

SQLite is extremely lightweight and stores the database locally with the application.

For example:

EXAMPLE / FLOW

Mobile application
Desktop application
Small local application

SQLite may be enough.

SQL Server is better suited when you need a centralized database serving many applications or users.

For example:

EXAMPLE / FLOW

```
User 1
v
User 2 -> SQL Server <- User 3
^
User 4
```

When Should You Use SQL Server?

SQL Server is a strong choice when you are developing applications that have structured business data and require reliability.

Examples include applications involving:

EXAMPLE / FLOW

Customers
Orders
Payments
Employees
Accounts
Inventory
Claims
Invoices
Transactions

It is especially worth considering when:

- You are developing with .NET / ASP.NET Core
- Your organization uses the Microsoft ecosystem
- You need strong transaction support
- You need strong security
- Multiple users access the database
- Data consistency is important
- The system contains important business data
- You require backup and recovery capabilities
- You expect the application to grow
- You need mature administration and monitoring tools
- You need an enterprise-grade relational database

For your planned stack:

EXAMPLE / FLOW

```
.NET Web API
+
Angular
+
Azure
```

SQL Server is a very useful database to learn.

When Should You Not Use SQL Server?

SQL Server may not be the best choice when its strengths are unnecessary or another database better matches your requirements.

For example, reconsider SQL Server when:

#

You Need a Tiny Embedded Database

For a small local desktop or mobile application:

EXAMPLE / FLOW

Application
v
Local database file

SQLite may be simpler.

#

Commercial Licensing Is a Major Concern

If your organization requires a completely open-source database platform and wants to avoid commercial SQL Server licensing, PostgreSQL or MySQL may be more appropriate.

#

Your Data Model Is Fundamentally Non-Relational

Some applications deal primarily with specialized forms of information for which another database category may be more appropriate.

You should choose the database based on the data and application requirements rather than using SQL Server automatically.

#

The Application Is Extremely Small

For a tiny program storing only temporary information, introducing a database server may add unnecessary complexity.

Why SQL Server Makes Sense for Your Learning Path

For the technologies you are preparing to learn:

EXAMPLE / FLOW

SQL Server
.NET
ASP.NET Core Web API
Angular
Azure

EXAMPLE / FLOW (cont.)

MCP
Agentic AI

SQL Server gives you a strong foundation for understanding how real business applications store and manage relational data.

Later, when you build applications, the flow may look approximately like:

EXAMPLE / FLOW

Angular
v
ASP.NET Core Web API
v
SQL Server

You don't need to understand that complete architecture yet.

For now, understand one thing clearly:

SQL Server is the software responsible for managing the databases and business data used by your application.

Simple Final Comparison

Remember these four statements:

EXAMPLE / FLOW

Database
= The organized data

DBMS
= Software used to manage databases

SQL
= Language used to communicate with relational databases

Microsoft SQL Server
= Microsoft's relational database management system

And the simplest definition to remember is:

Microsoft SQL Server is an RDBMS developed by Microsoft that is used to store, retrieve, update, secure, and manage relational data for applications.

#

Interview Answer

If an interviewer asks "What is SQL Server?", you can say:

SQL Server is a relational database management system developed by Microsoft. It is used to store, manage, retrieve, secure, and process application data. It uses T-SQL for interacting with databases and is commonly used for enterprise applications, especially with .NET and Microsoft technologies.

#

Final Point

Do not think:

EXAMPLE / FLOW

SQL Server is always the best database.

Think:

EXAMPLE / FLOW

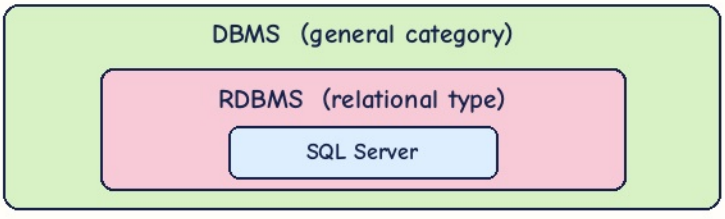
SQL Server is a strong choice
when its features match the application's requirements.

For enterprise applications built using C

and ASP.NET Core, it is one of the most important relational database systems to understand.

DBMS vs RDBMS

DBMS -> RDBMS -> SQL SERVER



Every RDBMS is a DBMS, but every DBMS is not necessarily an RDBMS.

The easiest way to understand this is:

DBMS is the general concept. RDBMS is a type of DBMS that stores related data mainly in tables.

EXAMPLE / FLOW

```
DBMS
|
+-- RDBMS
```

So:

Every RDBMS is a DBMS, but every DBMS is not necessarily an RDBMS.

#

What is DBMS?

DBMS stands for Database Management System.

It is software used to create, store, retrieve, update, delete, and manage data in databases.

Simple definition:

DBMS is software used to manage databases.

Examples of database-management systems include different systems that may organize data in different ways.

Conceptually:

EXAMPLE / FLOW

```
Application
v
DBMS
v
Database
```

The important point is that DBMS is the broader category.

#

What is RDBMS?

RDBMS stands for Relational Database Management System.

It is a type of DBMS that organizes data mainly in tables consisting of rows and columns, and those tables can be related to each other.

Simple definition:

RDBMS is a DBMS that stores and manages related data using tables.

For example:

#

Customers

CustomerId	Name
1	John
2	Priya

#

Orders

OrderId	CustomerId	Amount
101	1	500
102	1	250
103	2	700

Notice that both tables contain:

EXAMPLE / FLOW

CustomerId

This allows the customer data and order data to be related.

EXAMPLE / FLOW

```
Customers
|
| CustomerId
v
Orders
```

That relationship is the main idea behind the word Relational in RDBMS.

DBMS vs RDBMS

DBMS	RDBMS
Database Management System	Relational Database Management System
General category of database-management software	A specific type of DBMS
Data can be organized using different models	Data is primarily organized using relational tables
Relationships are not necessarily the main concept	Relationships between tables are fundamental
May use different structures depending on the DBMS type	Uses rows and columns in tables
May not provide the same level of relational constraints	Commonly supports keys and constraints to maintain relationships
Broader term	More specific term

The most important difference is:

EXAMPLE / FLOW

DBMS
=
Software that manages databases

RDBMS
=
DBMS that manages relational databases

Simple Example

Suppose you have:

EXAMPLE / FLOW

Students
Courses
Enrollments

A relational system can connect them:

EXAMPLE / FLOW

Students
|
v
Enrollments
^
|
Courses

For example:

#

Students

StudentId	Name
1	John
2	Priya

#

Courses

CourseId	CourseName
10	SQL
20	C

#

Enrollments

StudentId	CourseId
1	10
1	20
2	10

The data is divided into tables, and relationships connect those tables.

This is the type of data organization used by an RDBMS.

Where Does SQL Server Fit?

Microsoft SQL Server is an:

EXAMPLE / FLOW

RDBMS

And because an RDBMS is a type of DBMS:

EXAMPLE / FLOW

SQL Server
v
RDBMS
v
DBMS

Therefore, both statements are technically correct:

EXAMPLE / FLOW

SQL Server is a DBMS.

and:

EXAMPLE / FLOW

SQL Server is an RDBMS.

But RDBMS is more specific and accurate because SQL Server uses the relational model.

Other well-known relational database systems include:

EXAMPLE / FLOW

PostgreSQL
MySQL
Oracle Database

EXAMPLE / FLOW (cont.)

Microsoft SQL Server

Why Do We Use an RDBMS?

RDBMSs are very useful when application data is naturally related.

For example:

EXAMPLE / FLOW

Customer -> Orders
Employee -> Department
Student -> Courses
Patient -> Appointments
Product -> OrderItems

An RDBMS makes it easier to organize these relationships and maintain correct data.

It commonly provides capabilities such as:

- Tables
- Relationships
- Primary keys
- Foreign keys
- Constraints
- SQL querying
- Transactions
- Data integrity

We will study each of these concepts separately.

Advantages of RDBMS

#

Structured Data

Data is organized clearly using rows and columns.

#

Relationships

Different tables can be connected logically.

#

Reduced Duplication

Good relational design can reduce unnecessary repetition of data.

#

Data Integrity

Rules can be applied to prevent invalid or inconsistent data.

#

Powerful Querying

SQL can retrieve data from multiple related tables.

#

Transaction Support

Business operations can be performed reliably as transactions.

#

Security

Access to database information can be controlled.

Limitations of RDBMS

RDBMS systems are excellent for structured relational data, but they are not automatically the best solution for every type of application.

They can become more complex when:

- The data structure changes constantly.
- The information does not naturally fit a relational model.
- The application needs a very specialized non-relational storage model.
- A tiny application only needs very simple local storage.

The database technology should always be chosen according to application requirements.

Important Point to Remember

Do not think of DBMS and RDBMS as two completely unrelated things.

Think of them like this:

EXAMPLE / FLOW

```
Vehicle
|
+-- Car
```

A car is a vehicle, but every vehicle is not a car.

Similarly:

EXAMPLE / FLOW

```
DBMS
|
+-- RDBMS
```

An RDBMS is a DBMS, but every DBMS does not have to be relational.

Another simple formula is:

EXAMPLE / FLOW

DBMS
=
Database management software

RDBMS
=
Database management software
+
Relational tables
+
Relationships

#

Interview Answer

DBMS is software used to manage databases, while RDBMS is a type of DBMS based on the relational model, where data is mainly stored in tables containing rows and columns and relationships can be established between those tables. Every RDBMS is a DBMS, but every DBMS is not necessarily an RDBMS.

#

Final Memory Point

EXAMPLE / FLOW

DBMS -> General category

RDBMS -> Relational type of DBMS

SQL Server -> RDBMS

Example

Suppose an application uses a database called:

ShoppingDb

That database may store information related to:

EXAMPLE / FLOW

Customers
Products
Orders
Payments

All of this information belongs to the same shopping application, so it can be stored together inside the database.

Creating a Database in SQL Server

In SQL Server, a database can be created using the CREATE DATABASE statement.

SQL

```
CREATE DATABASE ShoppingDb;
GO
```

This creates a new database named:

ShoppingDb

To work with that database, we can use:

SQL

```
USE ShoppingDb;
GO
```

We can check the currently selected database using:

SQL

```
SELECT DB_NAME() AS CurrentDatabase;
```

What a Database Allows Us to Do

A database allows applications to:

EXAMPLE / FLOW

Store data
Retrieve data
Update data
Delete data
Organize data
Protect data
Maintain data consistently

Allow multiple users or applications to work with the same information

The common operations performed on stored data are often called CRUD:

SQL

Create

EXAMPLE / FLOW

Read
Update
Delete

We will study the SQL statements used for these operations separately.

Advantages

Using a database provides several benefits.

It keeps data organized and permanently stored. It makes information easier to search and update. It also provides mechanisms for maintaining data consistency, controlling access, and handling large amounts of information.

A properly designed database is much easier to maintain than storing application data in unrelated files or temporary variables.

Limitations

A database also requires proper design and management.

Poor database design can lead to duplicated data, incorrect information, slow performance, and difficult maintenance.

For very small programs that only use temporary values, a database may not be necessary.

For example:

int total = 10 + 20;

If total is only required temporarily while the program is running, there is usually no need to store it in a database.

Naming Best Practice

Database names should clearly describe their purpose.

Good examples:

EXAMPLE / FLOW

EmployeeDb
InventoryDb
ShoppingDb
FundraisingDb

Avoid unclear names such as:

EXAMPLE / FLOW

db1
newdb
test123
abc

A meaningful name makes the system easier to understand and maintain.

Important Point to Remember

A database is mainly used to store and manage related data permanently in an organized way.

A simple way to remember it is:

EXAMPLE / FLOW

Database
=
Organized Data
+
Permanent Storage
+
Data Management



Interview Definition

A concise interview answer is:

A database is an organized collection of related data stored electronically that allows applications to efficiently store, retrieve, update, delete, and manage information.